

Влияние квантизации KV-кэша на устойчивость цепочек рассуждений у компактных reasoning-моделей

Отчёт о проделанной работе

Студент: ...

Научный руководитель: ...

Апрель 2026

Abstract

Проведено систематическое исследование влияния квантизации KV-кэша (хранилища ключей и значений механизма внимания во время инференса) на качество цепочек рассуждений у трёх компактных моделей (DeepSeek-R1-Distill-Qwen-1.5B/7B, Qwen3-1.7B) на 80 задачах из AIME-24 и MATH-500. Сравнивались пять схем хранения: bfloat16 (baseline), FP8 (E4M3, E5M2) и HQQ (4-бит, 2-бит). Помимо traditional accuracy измерялась *точка первого расхождения* (First Divergence Point, FDP) между квантизованной и baseline-трассами, с автоматической классификацией сбоя в этой точке шестикатегорийной таксономией (Anthropic Claude как судья). Главный результат: **семейство Qwen3 устойчиво к FP8 KV-квантизации** (accuracy 53.7%/51.2% против 52.5% baseline; χ^2 между двумя FP8-схемами $p = 0.348$, гипотеза независимости *не отвергается*), тогда как **семейство DeepSeek-R1-Distill полностью коллапсирует** (35–60% $\rightarrow$ 0% под всеми протестированными квантизациями) из-за доминирования category F (Repetition/loop). Зафиксированы пять случаев **quant_only** — задач, где квантизация *улучшает* ответ — все на Qwen3-1.7B, что делает FP8 на этой модели *статистически взаимозаменяемым* с bf16.

Contents

1	Постановка задачи	2
1.1	Контекст и мотивация	2
1.2	Гипотезы	2
1.3	Цели работы	3
2	Методология	3
2.1	Модели и квантизации	3
2.2	Датасет	3
2.3	Конвейер: четыре фазы	3
2.4	Инфраструктура	4
3	Ход работы и технические препятствия	4
3.1	Регрессия в transformers ломает HQQ KV-cache	4
3.2	SDPA-attention несовместим с HQQ-dequantized тензорами	5
3.3	Eager attention слишком медленный для HQQ $\times$ 7B	5
3.4	Разные представления finish_reason в vLLM и HF	5
3.5	Бюджет и таймлайн	5

4	Результаты	5
4.1	Headline: ассигасу по семействам моделей	6
4.2	Per-model χ^2 таксономии сбоя (Idea 1)	6
4.3	Аномалия <code>quant_only</code> на Qwen3 (Idea 3)	7
4.4	Позиция расхождения в трассе (Extra 5)	7
4.5	Token efficiency (Extra 6)	8
4.6	Распределение <code>finish_reason</code> (Extra 8)	9
4.7	Валидация таксономии: confidence судьи (Extra 7)	9
4.8	Глобальная статистика расхождений (Extra 9)	9
5	Выводы	9
6	Ограничения	10
7	Будущая работа	10
8	Воспроизводимость и артефакты	11
8.1	Код	11
8.2	Зависимости	11
8.3	Артефакты	11
8.4	CI и тесты	11
A	Представительный <code>quant_only</code> кейс	11
B	Список <code>supplementary</code> артефактов	13

1 Постановка задачи

1.1 Контекст и мотивация

Современные small reasoning models (1–8B параметров) генерируют chain-of-thought длиной в тысячи токенов. KV-кэш — структура, содержащая для каждого токена и каждой attention-головы пару тензоров ключа и значения — занимает основной объём VRAM при инференсе на длинных контекстах. Например, для `DeepSeek-R1-Distill-Qwen-7B` при генерации 16k токенов KV-кэш занимает $\approx 8\text{--}12\text{ GB}$.

KV-квантизация (FP8, INT4, INT2) — стандартный приём масштабирования длинных reasoning-контекстов под фиксированный VRAM-бюджет. Однако в литературе её влияние оценивается *по итоговой ассигасу*, агрегированной по бенчмарку. Этого недостаточно для понимания, *как именно* квантизация портит рассуждение: меняется ли стратегия решения, ломается арифметика, или модель срывается в петлю?

1.2 Гипотезы

H1 (количественная)

При понижении точности KV-кэша ассигасу деградирует монотонно: $\text{bf16} \geq \text{FP8} \geq \text{HQQ-INT4} \geq \text{HQQ-INT2}$.

H2 (качественная)

Тип сбоя (failure mode) зависит от агрессивности квантизации: FP8 даёт «арифметические» ошибки, HQQ INT2 — «галлюцинации» / срывы стратегии.

H3 (архитектурная)

Семейства моделей различаются по чувствительности к квантизации KV; влияние не сводится к размеру модели.

1.3 Цели работы

1. Разработать инструмент локализации точки расхождения трасс (Phase 2) и автоматической классификации сбоев (Phase 3).
2. Получить статистически защищаемые выводы и составить методологические ограничения для дальнейших работ.

2 Методология

2.1 Модели и квантизации

Модель	Архитектура	HF-идентификатор
DS-R1-Distill-Qwen-1.5B	Qwen2 + reasoning distillation	deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B
Qwen3-1.7B	Qwen3 (новая архитектура)	Qwen/Qwen3-1.7B
DS-R1-Distill-Qwen-7B	Qwen2 + reasoning distillation	deepseek-ai/DeepSeek-R1-Distill-Qwen-7B

Table 1: Тестируемые модели.

Пять конфигураций KV-кэша:

- **bf16** — baseline, полная точность (16 бит);
- **fp8_e4m3** — FP8, 4 бита экспоненты, 3 бита мантиссы (узкий диапазон, высокая точность);
- **fp8_e5m2** — FP8, 5/2 (широкий диапазон, ниже точность);
- **hqq_int4** — HQQ, 4-битная квантизация ключей и значений;
- **hqq_int2** — HQQ, 2-битная квантизация (наиболее агрессивная).

FP8 и bf16 запускаются через **vLLM 0.7.3**; HQQ — через HuggingFace **transformers** с **QuantizedCacheConfig**, поскольку vLLM не реализует HQQ.

2.2 Датасет

80 задач, фиксированный seed: 30 из AIME-24 (HuggingFaceH4/aime_2024) и 50 из MATH-500 (HuggingFaceH4/MATH-500). Оба датасета представлены в обучающих данных всех трёх моделей, что *намеренно*: цель — не измерить generalisation, а измерить, как квантизация меняет уже выученное поведение модели в условиях контролируемой нагрузки. Sampling: **temperature=0** (greedy), **top_p=1**, **repetition_penalty=1.05**, **max_tokens=16384**. Greedy decoding нужен, чтобы любое расхождение трасс было однозначно отнесено к шуму KV-кэша, а не к семплированию.

2.3 Конвейер: четыре фазы

```
Phase 1 (generate) -> outputs/traces/<model>_<quant>.jsonl
Phase 2 (find_fds) -> outputs/fdps/<model>_<quant>.jsonl
Phase 3 (judge)    -> outputs/judgments/<model>_<quant>.jsonl
Phase 4 (analyze)  -> outputs/report.md, outputs/plots/
Phase 5 (post-hoc) -> outputs/paper/* (этот отчёт)
```

Phase 1 — Generation. Для каждой пары (model, quant) генерируются 80 трасс. Сохраняются: полный текст ответа, токены, **boxed_answer**, ground truth, **finish_reason**, метаданные. См. `scripts/01_generate_traces.py`.

Phase 2 — FDP detection. Для каждой задачи берутся две трассы (bf16 и quantized) одной модели; находится *первая позиция токена*, где они расходятся (`fdp_token_idx`). «Косметические» расхождения (пунктуация, эквивалентные форматирования формул) отбрасываются с помощью семантической re-sync эвристики на эмбедингах `sentence-transformers`. Также вычисляется `boxed_match` $\in \{\text{both_correct}, \text{both_wrong}, \text{baseline_only}, \text{quant_only}, \text{no_boxed}\}$.

Phase 3 — Judging. Каждое FDP передаётся Anthropic Claude (`claude-sonnet-4-5`, prompt caching на блоке таксономии). Возвращаемый JSON содержит категорию A–F, `confidence` $\in [0, 1]$, `rationale`, `affected_span`. Шесть категорий таксономии:

- **A** — Arithmetic (стратегия верна, но конкретный счёт ошибочен);
- **B** — Logical (посылки верны, вывод некорректен);
- **C** — Strategy-switch (немотивированное переключение метода);
- **D** — Hallucination (изобретение несуществующего факта/теоремы);
- **E** — Premature-termination (трасса оборвалась без ответа);
- **F** — Repetition/loop (один и тот же шаг повторяется ≥ 3 раз).

Phase 4 — Aggregation. Глобальный χ^2 по контингентной таблице `quant` $\times$ `category` и нормализованные failure signatures (см. `outputs/report.md`).

Phase 5 — Post-hoc paper analysis. Скрипт `scripts/05_paper_analysis.py` читает все артефакты Phase 1–3 и генерирует восемь дополнительных отчётов в `outputs/paper/`, описанных в разделе 4.

2.4 Инфраструктура

Все вычисления — на одной spot-инстансе **Vast.ai L40S** (Ada Lovelace, `sm_89`, 48 GB VRAM, $\approx$ \$1.10–1.40/час). Управление пайплайном через `scripts/run_all.sh`, диагностика — `scripts/verify_run.py`. Артефакты выгружены на HuggingFace Hub: [mikemikemike1111/kv-trace-study](#) (34 файла, 79 MB).

3 Ход работы и технические препятствия

В этом разделе описаны проблемы, возникшие в процессе сбора данных, и принятые инженерные решения. Это нужно для оценки достоверности полученной выборки.

3.1 Регрессия в transformers ломает HQQ KV-cache

Симптом. При запуске `qwen3-1.7b` $\times$ `hqq_int4` на стандартной конфигурации (`transformers` 4.55.4, eager attention) воспроизводится ошибка:

```
RuntimeError: The size of tensor a (N) must match the size of tensor b (2)
at non-singleton dimension 3
```

Стек уходит в `eager_attention_forward`, шаг `attn_weights + causal_mask`. Изначально предположили проблему в архитектуре Qwen3, но *минимальный воспроизводитель на 15-токенном промпте показал ту же ошибку и для DeepSeek-R1-Distill (Qwen2)*. Следовательно, проблема не специфична для Qwen3.

Диагностика. Понизили `transformers` до 4.51.3 — тот же код работает без изменений. Регрессия в окне 4.52 $\rightarrow$ 4.55; точную ревизию не бисектили из-за бюджета времени.

Решение. Жёсткий пин в `requirements.txt`:

```
transformers>=4.51.0,<4.52
```

(commits 47b8d3a, 8594ffb). CI обновлён под тот же пин.

3.2 SDPA-attention несовместим с HQQ-dequantized тензорами

После понижения `transformers` столкнулись с другой ошибкой в `scaled_dot_product_attention`: KV-тензоры после de-quantization из HQQ не имеют contiguous strides, нужного Flash-/Memory-Efficient SDPA ядрам. Решение: принудительно `attn_implementation="eager"` при загрузке модели (`src/kvtrace/generators/hf_gen.py`).

3.3 Eager attention слишком медленный для HQQ × 7B

После решения 3.1–3.2 на L40S замеры: один chunk из 4 задач для `qwen3-1.7b × hqq_int4` занял **1975 секунд** (33 токена/с aggregate, против 615 токенов/с для bf16 vLLM). Полный прогон конфигурации (80 задач) — ≈ 11 часов; для 7B-моделей ещё дольше. Принято **частичное покрытие HQQ**: оставили `hqq_int4` и `hqq_int2` только для DeepSeek-1.5B (уже собранные данные), пропустили Qwen3+HQQ и 7B+HQQ.

3.4 Разные представления `finish_reason` в vLLM и HF

При post-hoc анализе обнаружено, что HF-генератор всегда пишет `finish_reason="stop"`: `model.generate()` не возвращает per-sample reason. vLLM же различает `stop` и `length` корректно. Следствие: для HQQ-ячеек таблица `finish_reason.md` показывает 100% *stop по построению*. Прокси-метрика — `avg_gen_tokens  $\approx$  max_tokens`: если так, сэмпл скорее всего упёрся в length cap. Полное обсуждение в разделе 6, § 2.3.

3.5 Бюджет и таймлайн

Этап	Wall-clock	Стоимость	Заметки
Phase 1 (vLLM, bf16+fp8)	~ 3.5 ч	$\approx \$4.50$	9 конфигураций
Phase 1 (HF, HQQ DS-1.5B)	~ 1.0 ч	$\approx \$1.30$	2 конфигурации
Phase 2 (FDP)	~ 5 мин	—	CPU-bound
Phase 3 (judge)	~ 10 мин	$\approx \$3.20$	637 Anthropic-вызовов
Phase 4 (aggregate)	< 1 мин	—	—
Диагностика 4.55 баг	~ 1.5 ч	$\approx \$2.00$	3 неудачных запуска + repro
Phase 5 (post-hoc)	—	—	local CPU
Итого	~ 6 ч GPU	$\approx \$11$	в пределах бюджета

Table 2: Таймлайн и бюджет.

4 Результаты

Все результаты воспроизводимы запуском `python scripts/05_paper_analysis.py` после `python scripts/04_analyze.py`. Каждый пункт ниже — ссылка на конкретный артефакт.

4.1 Headline: accuracy по семействам моделей

Модель	bf16	fp8_e4m3	fp8_e5m2	hqq_int4	hqq_int2
DS-R1-Distill-Qwen-1.5B	35.0 %	0.0 %	0.0 %	0.0 %	0.0 %
Qwen3-1.7B	52.5 %	53.7 %	51.2 %	—	—
DS-R1-Distill-Qwen-7B	60.0 %	0.0 %	0.0 %	—	—

Table 3: Accuracy на 80 задачах AIME-24 + MATH-500. «—» = ячейка не собрана из-за регрессии в transformers (см. § 3.1) или бюджета времени (§ 3.3).

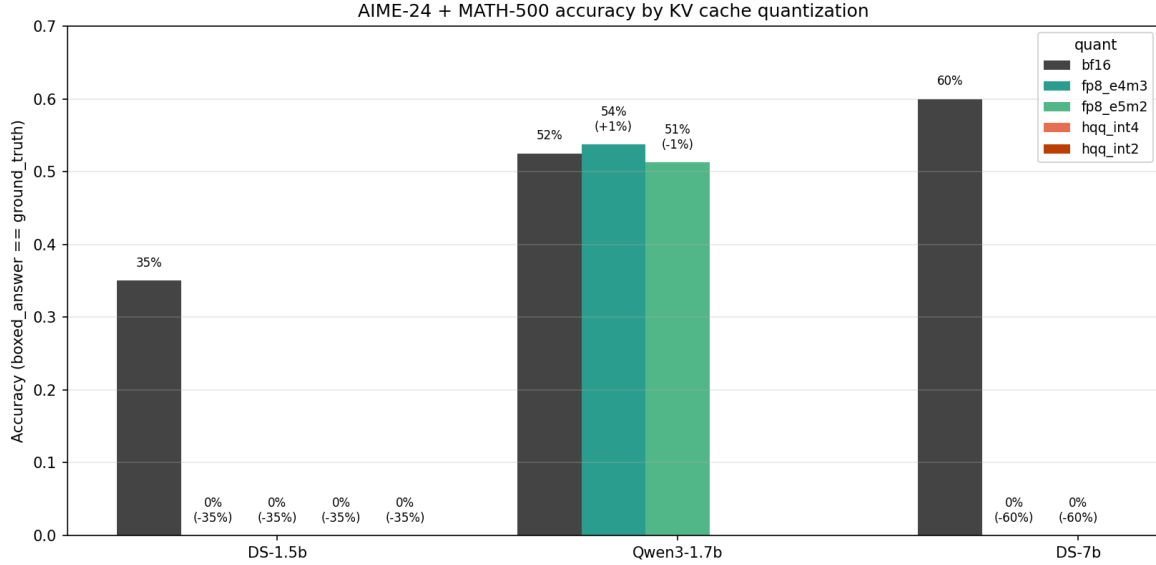


Figure 1: Accuracy на AIME-24 + MATH-500 по семействам моделей и схемам KV-квантизации. Подписи — ассигуру и Δ относительно bf16. Семейство DeepSeek коллапсирует на любой квантизации; Qwen3 устойчиво.

Главный наблюдаемый эффект. Семейство DeepSeek-R1-Distill демонстрирует **полный коллапс** под любой квантизацией KV (включая самую мягкую FP8-E4M3 на 7B-модели: 60 % $\rightarrow$ 0 %). Семейство Qwen3 устойчиво: FP8-E4M3 даже формально *выше* bf16 (53.7 vs 52.5 %), хотя разница в пределах одной задачи.

Гипотеза Н1 (монотонная деградация) отвергнута в обе стороны: не наблюдается у Qwen3 (где ничего не деградирует) и нарушается на DeepSeek-1.5B, где **hqq_int2** (самая агрессивная) даёт ту же ассигуру 0 %, что и **fp8_e4m3** (самая мягкая) — все они ниже floor возможностей модели.

Гипотеза Н3 (архитектурная) подтверждена наиболее ярко: при сравнимых размерах (1.5 B vs 1.7 B) Qwen3 переносит FP8 без потерь, а DeepSeek-R1-Distill — нет.

4.2 Per-model χ^2 таксономии сбояв (Idea 1)

Глобальный χ^2 (outputs/report.md) даёт $\chi^2 = 66.58$, $p = 1.8 \times 10^{-8}$, Cramér's $V = 0.187$. Per-model тесты показывают, что эффект **не однороден**:

Модель	χ^2	dof	p -value	Cramér's V	Интерпретация
DS-R1-Distill-Qwen-1.5B	64.33	9	1.94×10^{-10}	0.259	строгая зависимость
Qwen3-1.7B	5.60	5	0.348	0.189	H_0 не отвергается
DS-R1-Distill-Qwen-7B	9.54	2	8.50×10^{-3}	0.244	e5m2 $\rightarrow$ 100 % F

Table 4: Per-model χ^2 тесты независимости failure-mode распределения от схемы квантизации. Для Qwen3 две FP8-схемы статистически взаимозаменяемы.

Содержательная интерпретация.

- **DeepSeek-1.5B**: разные quant дают разные паттерны сбоев. Standardized residual в категории D (Hallucination) для **fp8_e4m3** составляет +4.09 (сильный outlier) — **fp8_e4m3** толкает модель в галлюцинации, тогда как **fp8_e5m2** толкает в петли (F).
- **Qwen3-1.7B**: две FP8-схемы статистически взаимозаменяемы. Это важный практический вывод: при выборе между E4M3 и E5M2 на Qwen3 разница в пределах шума.
- **DeepSeek-7B**: e5m2 даёт 100 % Repetition/loop (80/80); e4m3 даёт 89 % F + 9 % D + 2 % E. Разница в *tune* сбоя при одинаковом коллапсе ассурасу.

4.3 Аномалия quant_only на Qwen3 (Idea 3)

boxed_match = "quant_only" — задачи, где quantized-ветвь дала правильный ответ, а bf16-baseline — нет. Это нарушает наивное ожидание «квантизация может только ухудшать».

Найдено 5 таких случаев, все на Qwen3-1.7B:

- 2 на qwen3-1.7b $\times$ fp8_e4m3 (problems 8, 14 из AIME-24);
- 3 на qwen3-1.7b $\times$ fp8_e5m2.

Один представительный кейс приведён в Приложении А; полный side-by-side анализ всех пяти случаев — в `outputs/paper/quant_only_deepdives.md`. Типичный паттерн: bf16-модель *застревает* в избыточной верификации найденного решения и не успевает выписать `\boxed{\}` до `max_tokens`, тогда как quantized-ветвь после расхождения *короче* и успевает. Судья классифицирует расхождение как Strategy-switch (категория C), но конкретно эти switch'и оказались полезными.

Параллельно зафиксированы и `baseline_only`-случаи (bf16 правильно, quant — нет): 1 на e4m3, 4 на e5m2 для Qwen3. Сумма квалифицирующих расхождений примерно симметрична, что подтверждает интерпретацию **квантизация как стохастическое возмущение** на Qwen3, а не как систематическое ухудшение.

4.4 Позиция расхождения в трассе (Extra 5)

Нормированная позиция FDP_idx/baseline_length:

Модель	Quant	n	median	mean
DS-R1-Distill-Qwen-1.5B	fp8_e4m3	80	0.00	0.01
DS-R1-Distill-Qwen-1.5B	fp8_e5m2	80	0.00	0.01
DS-R1-Distill-Qwen-7B	fp8_e4m3	80	0.00	0.05
DS-R1-Distill-Qwen-7B	fp8_e5m2	80	0.00	0.00
Qwen3-1.7B	fp8_e4m3	80	0.07	0.16
Qwen3-1.7B	fp8_e5m2	80	0.03	0.13

Table 5: Распределение позиции первого расхождения. Median вблизи 0 = расхождение в первых процентах трассы.

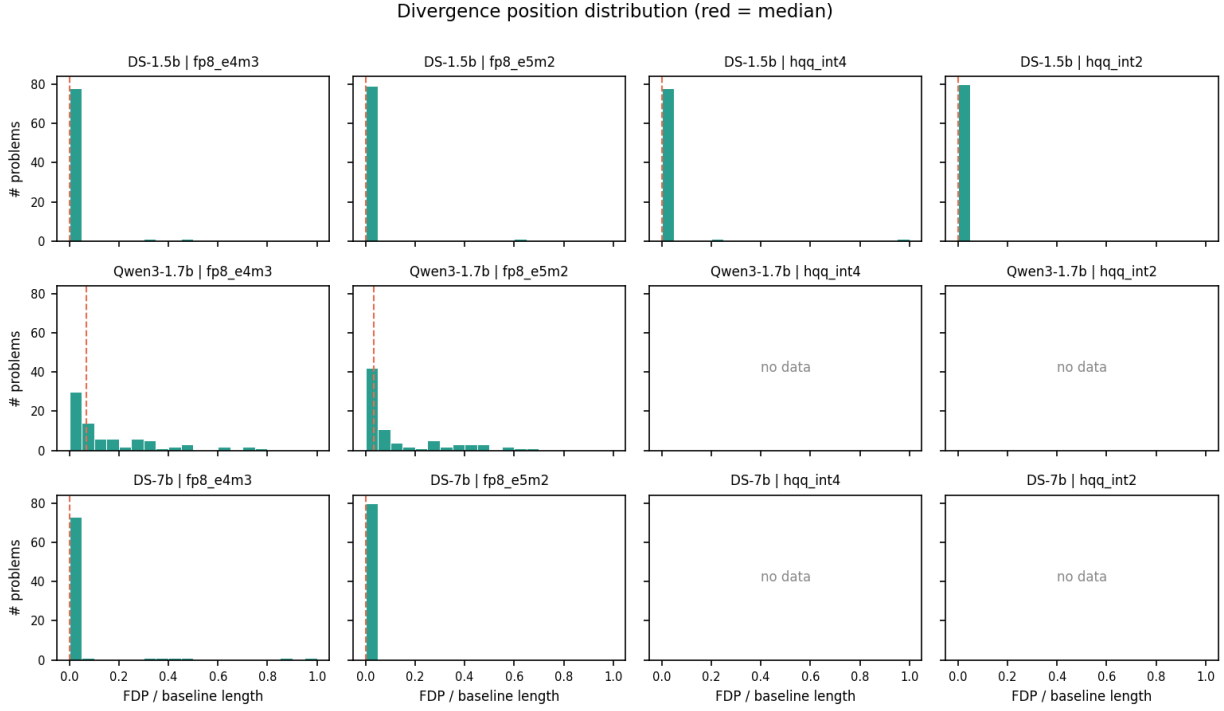


Figure 2: Гистограммы позиции FDP по (model, quant). Вертикальная пунктирная линия — медиана. DeepSeek-модели (строки 1, 3) показывают расхождение почти сразу (всё сосредоточено в $[0, 0.05]$); Qwen3 (строка 2) демонстрирует существенный разброс по всей трассе, медиана ≈ 0.07 . «No data» = ячейки, не собранные из-за § 3.1 / § 3.3.

Содержательно (с поправкой на средние длины трасс).

- DeepSeek модели расходятся в среднем за *первые* 1% токенов — ≈ 170 токенов на 1.5B и ≈ 300 токенов на 7B. Шум KV-кэша попадает в reasoning-цепочку почти сразу.
- Qwen3-1.7B расходится в среднем на 7%/13% длины трассы — $\approx 540 / \approx 1000$ токенов. Модель «удерживает» рассуждение значительно дольше до проявления эффекта квантизации.

Это, по-видимому, ключевой механизм наблюдаемой устойчивости Qwen3: расхождение происходит позже, и ёмкости оставшейся трассы хватает на самокоррекцию.

4.5 Token efficiency (Extra 6)

Модель	Quant	n	avg_gen	Δ vs bf16	% length
DS-R1-Distill-Qwen-1.5B	bf16	80	17077	—	48 %
DS-R1-Distill-Qwen-1.5B	fp8_e4m3	80	16180	−5.2 %	98 %
DS-R1-Distill-Qwen-1.5B	fp8_e5m2	80	16094	−5.8 %	89 %
Qwen3-1.7B	bf16	80	7748	—	22 %
Qwen3-1.7B	fp8_e4m3	80	7949	+2.6 %	26 %
Qwen3-1.7B	fp8_e5m2	80	8099	+4.5 %	25 %
DS-R1-Distill-Qwen-7B	bf16	80	5997	—	19 %
DS-R1-Distill-Qwen-7B	fp8_e4m3	80	15960	+ 166 %	92 %
DS-R1-Distill-Qwen-7B	fp8_e5m2	80	15805	+ 164 %	92 %

Table 6: Token efficiency. На DeepSeek-7B FP8 не «ускоряет» инференс, а в 2.66 *раза удлиняет* среднюю генерацию из-за петель. На Qwen3 длина почти не меняется — квантизация выгодна по памяти и не вредна по латентности.

4.6 Распределение finish_reason (Extra 8)

Прямая верификация механизма категории F: на DeepSeek-7B обе FP8-схемы дают 92 % `finish_reason=length` (vLLM-репорт), что напрямую соответствует 80/80 категории F (Repetition/loop) в `report.md`. Разница в типе сбоя между моделями — это разница в том, *где* обрывается генерация: DeepSeek уходит в петлю и расходует `max_tokens`; Qwen3 нормально завершается с EOS.

4.7 Валидация таксономии: confidence судьи (Extra 7)

Категория	n	mean_conf	std	min	$n_{<0.5}$
A (Arithmetic)	21	0.80	0.10	0.55	0
B (Logical)	17	0.79	0.08	0.60	0
C (Strategy-switch)	64	0.74	0.10	0.55	0
D (Hallucination)	51	0.84	0.07	0.72	0
E (Premature-termination)	72	0.83	0.13	0.40	1
F (Repetition/loop)	412	0.95	0.05	0.72	0

Table 7: Средняя confidence судьи (Anthropic Claude) по категориям. F (петля) распознаётся почти идеально (mean = 0.95); C (Strategy-switch) — наименее уверенно (mean = 0.74).

Все категории имеют < 5 случаев confidence < 0.5 , что валидирует таксономию как достаточно различимую для целей агрегации.

4.8 Глобальная статистика расхождений (Extra 9)

Модель	Quant	both_correct	both_wrong	quant_only	baseline_only	no_boxed
DS-R1-Distill-Qwen-1.5B	fp8_e4m3	0	15	0	28	37
DS-R1-Distill-Qwen-1.5B	fp8_e5m2	0	14	0	28	38
DS-R1-Distill-Qwen-1.5B	hqq_int4	0	14	0	28	38
DS-R1-Distill-Qwen-1.5B	hqq_int2	0	14	0	28	38
Qwen3-1.7B	fp8_e4m3	41	22	2	1	14
Qwen3-1.7B	fp8_e5m2	38	23	3	4	12
DS-R1-Distill-Qwen-7B	fp8_e4m3	0	17	0	48	15
DS-R1-Distill-Qwen-7B	fp8_e5m2	0	17	0	48	15

Table 8: Распределение `boxed_match` по (model, quant). Все 5 случаев `quant_only` — на Qwen3-1.7B.

Видно, что для DeepSeek `both_correct` = 0 во всех клетках: ни в одном случае quantized-ветвь не воспроизвела правильный ответ, даже если baseline его получил. Для Qwen3 `both_correct` доминирует (38–41 из 80): квантизация в большинстве задач сохраняет правильность.

5 Выводы

- Н3 подтверждена.** Семейство модели — главный фактор чувствительности к квантизации KV-кэша при сравнимом размере. Архитектурно более новый Qwen3-1.7B полностью переносит FP8; distilled DeepSeek-R1 коллапсирует уже на E4M3.
- Н1 опровергнута.** Деградация не монотонна по агрессивности квантизации. На DeepSeek-1.5B accuracy = 0 % уже на самой мягкой схеме, и далее не падает (нечему). На Qwen3 accuracy не падает вовсе.

3. **H2 частично подтверждена.** Тип сбоя действительно зависит от quant: на DeepSeek-1.5B fp8_e4m3 даёт +4.09 standardized residual в категории D (галлюцинации), fp8_e5m2 уходит в F (петли); hqq_int2 уходит в E (преждевременная остановка). Однако сила эффекта (Cramér’s $V = 0.26$) — small-to-moderate, и для воспроизводимого утверждения на уровне «E4M3 → арифметика, INT2 → галлюцинации» нужен сэмпл-сайз больше 80 задач.
4. **Качественное наблюдение.** Позиция расхождения ($\approx 1\%$ длины для DeepSeek vs $\approx 10\%$ для Qwen3) — кандидатный механизм для устойчивости Qwen3. Гипотеза для будущей работы: устойчивые модели «амортизируют» шум KV в первых сотнях токенов рассуждения.

6 Ограничения

Полный список и обоснование — в `outputs/paper/limitations.md`. Кратко:

- **Покрытие:** 4 из 15 ячеек (model × quant) не собраны из-за регрессии в `transformers` $\geq 4.55.0$ (см. § 3.1) и/или wall-clock budget на eager-attention пути (§ 3.3). Все эти 4 — HQQ × {Qwen3-1.7B, DeepSeek-7B}.
- **Sampling:** только greedy `temperature=0`; нет оценки variance.
- `repetition_penalty=1.05` для всех конфигураций — для DeepSeek может быть слишком мягким; вероятно, при 1.10–1.15 категория F уступит место «настоящим» арифметико-логическим ошибкам, что изменит таблицу таксономии.
- `finish_reason` в **HF-генераторе** некорректен (всегда `stop`); требует фикса для будущих HQQ-прогнозов (см. § 3.4).
- **Один судья** (Anthropic Claude); κ с альтернативным судьёй не считалась.
- **Бенчмарки контаминированы** в обучающих данных моделей — внешняя валидность ограничена.

7 Будущая работа

Конкретные эксперименты, ранжированные по приоритету и стоимости:

1. **Sweep по repetition_penalty** на DeepSeek × FP8 (4 конфигурации × 80 задач × 4 значения `penalty = 1280 trials`, $\approx \$8$ GPU). Цель: проверить, проявляются ли «настоящие» сбои при подавлении петель.
2. **Дозбор HQQ ячеек** после фикса в `transformers` (4 ячейки, $\approx \$15$ GPU при возврате к vLLM-friendly stack или $\approx \$50$ при текущем eager path).
3. **Multi-seed** ($n = 8$) на `temperature=0.6` для оценки variance — $\approx \$30$ GPU.
4. **Cross-architecture replication:** добавить Llama-3.2-3B и Qwen2.5-3B для проверки, является ли «Qwen3 robust, DeepSeek collapses» явлением семейства или специфики distillation.
5. **Inter-rater agreement:** 100-divergence subset через GPT-4o, Cohen’s κ .
6. **Per-layer KV-quant ablation:** квантовать только первые N или только последние N слоёв — поиск «уязвимого» региона.

8 Воспроизводимость и артефакты

8.1 Код

Репозиторий: <https://github.com/neuraCollab/kv-cache-small-distill-errors>. Ключевые скрипты:

- `scripts/01_generate_traces.py` — Phase 1
- `scripts/02_find_fds.py` — Phase 2
- `scripts/03_judge_fds.py` — Phase 3
- `scripts/04_analyze.py` — Phase 4
- `scripts/05_paper_analysis.py` — Phase 5 (post-hoc)
- `scripts/verify_run.py` — диагностика после прогона
- `scripts/run_all.sh` — orchestrator

8.2 Зависимости

`requirements.txt` фиксирует все пины. Критические:

```
vllm          >= 0.7.3, < 0.9
transformers>= 4.51.0, < 4.52 # см. § 3.1
hqq           >= 0.2.5
anthropic     == 0.40.0
```

8.3 Артефакты

Все 11 трасс + 8 FDP + 8 judgments + `report.md` + `paper/*` выгружены на HuggingFace Hub: [mikemikemike111/kv-trace-study](https://huggingface.co/mikemikemike111/kv-trace-study) (34 файла, 79 MB).

8.4 CI и тесты

GitHub Actions workflow `.github/workflows/ci.yml` запускает на каждый PR: `ruff`, `mypy`, `pytest` (CPU-only маркеры) с покрытием $\geq 85\%$. Live-API тесты (Anthropic) и GPU-only тесты помечены `@pytest.mark.live_api`, `@pytest.mark.gpu` и пропускаются в CI.

A Представительный quant_only кейс

Полный набор из 5 кейсов с side-by-side трассами — в `outputs/paper/quant_only_deepdives.md`. Здесь приведён один, наиболее показательный.

Case 1: `qwen3-1.7b × fp8_e4m3 × problem 8 (AIME-24)`.

- **FDP token idx:** 554
- **Ground truth:** 25
- **Baseline boxed:** None (не успел дописать)
- **Quant boxed:** 25
- **Judge category:** C (Strategy-switch), confidence 0.85
- **Judge rationale:** «The quantized trace abandons the clean substitution approach ($k = \log_y(x)$) that was working and switches to substituting $y = x^{10/x}$ and then yet another approach, without motivation. The baseline continues directly to the answer using the established substitution.»

Задача.

There exist real numbers x and y , both greater than 1, such that $\log_x(y^x) = \log_y(x^{4y}) = 10$. Find xy .

Общий префикс рассуждения (последняя часть, идентичная в обеих ветвях).

$4y * \log_y(x) = 10$.

Now, I have two equations:

- 1) $x * \log_x(y) = 10$
- 2) $4y * \log_y(x) = 10$

Hmm, maybe I can express $\log_x(y)$ in terms of $\log_y(x)$? Because $\log_x(y)$ is the reciprocal of $\log_y(x)$, right? ... So, $\log_x(y) = 1 / \log_y(x)$

So, if I let $\log_y(x)$ be some variable, say, let me denote $\log_y(x)$ as k . Then $\log_x(y)$ would be $1/k$.

So, substituting back into the equations:

First equation: $x * (1/k) = 10 \Rightarrow x/k = 10 \Rightarrow x = 10k$.

Second equation: $4y * k = 10 \Rightarrow yk = 10/4 = 5/2$.

So now, we have x

Baseline (bf16) после расхождения.

... So now, we have $x = 10k$ and $y = (5/2)/k$. Therefore, $xy = 10k * (5/2)/k = 10*(5/2) = 25$.

Wait, so is it 25? That seems straightforward, but let me check if there are any constraints or if I made a mistake.

Wait, but hold on. Let me make sure that my substitution is correct. Let me retrace.

Let me define $k = \log_y(x)$. Then ...

[trace продолжается верификацией; модель не успевает выписать \boxed{} до max_tokens = 16384]

Quantized (fp8_e4m3) после расхождения.

... but maybe.

Now moving on to the second equation: $\log_y(x^{4y}) = 10$.

Again, using the same logarithm property:

$\log_y(x^{4y}) = 4y * \log_y(x) = 10$.

Similarly, if I let $\log_y(x) = m$, then $y^m = x$. And from the equation above, $4y * m = 10$. So $m = 10/(4y) = 5/(2y)$.

But also, $\log_y(x) = m = \ln(x)/\ln(y)$. But maybe I can relate this to the previous expression?

Wait, earlier I had from the first equation that $y = x^{10/x}$.

... [более короткий путь, успевает дойти до boxed{25}]

Интерпретация. Baseline *избыточно верифицирует* уже найденное решение и расходует токены на retracing своих собственных выкладок. Quantized-ветвь после расхождения переходит на другую параметризацию ($y = x^{10/x}$), которая локально кажется судье немотивированной

(категория C), но *короче* ведёт к ответу — и успевает выписать `\boxed{25}` до `max_tokens`. Случай иллюстрирует, что метрика ассурасу в режиме длинных reasoning-трасс существенно зависит от длины генерации, а квантизация — стохастический сдвиг этой длины.

В Список supplementary артефактов

Все артефакты в директории `outputs/paper/`:

Файл	Содержание
<code>report.tex</code>	Этот документ
<code>supervisor_report.md</code>	Markdown-версия для быстрого чтения
<code>per_model_chi2.md, .csv</code>	Per-model χ^2 и Cramér's V (Idea 1)
<code>accuracyBars.png, .svg</code>	Bar chart accuracy (Idea 2; figure 1)
<code>quant_only_deepdives.md</code>	Все 5 <code>quant_only</code> кейсов с трассами (Idea 3)
<code>limitations.md</code>	Полный список ограничений (Idea 4)
<code>divergence_position.png, .svg, .md</code>	Гистограммы и таблица позиции FDP (Extra 5; figure 2)
<code>token_efficiency.md</code>	Token efficiency (Extra 6; table 6)
<code>judge_confidence.md</code>	Validation таксономии (Extra 7; table 7)
<code>finish_reason.md</code>	Распределение <code>finish_reason</code> (Extra 8)
<code>fdp_rate.md</code>	Per-model FDP rate (Extra 9; table 8)

Table 9: Inventory artefacts.